

Practical : - 10

Aim : - To interface Fingerprint Sensor (R307S) and LCD Display with Raspberry Pi.

Part I:Connection

1. LCD Display:

Pin No. 1 LCD Display (GND) = Pin no. 6 of Raspberry Pi (GND)

Pin No. 2 LCD Display (VCC) = Pin no. 2 of Raspberry Pi (5V)

Pin No. 3 LCD Display (SDA) = Pin no. 3 of Raspberry Pi (SDA)

Pin No. 4 LCD Display (SCL) = Pin no. 5 of Raspberry Pi (SCL)

2. Fingerprint Sensor:

Pin No. 1 Fingerprint Sensor(VCC) = Pin no. 4 of Raspberry Pi (5V)

Pin No. 2 Fingerprint Sensor (GND) = Pin no. 9 of Raspberry Pi (GND)

Pin No. 3 Fingerprint Sensor (TXD) = Pin no. 10 of Raspberry Pi (RXD)

Pin No. 4 Fingerprint Sensor (RXD) = Pin no. 8 of Raspberry Pi (TXD)

Pin No. 5 Fingerprint Sensor (Touch Sense) = No connection (X)

Pin No. 6 Fingerprint Sensor (Touch Sense Power) = No connection (X)

Part II : - Library Installation

The Library Has Been Already Installed In The Previous Fingerprint Sensor and LCD Display Practical

Part III : - Code & Output

```
fingerprint+Led.py X
1 #!/usr/bin/env python3
2 """
3 R3075: Fingerprint Attendance/ID Tool
4 -----
5 All-in-one script:
6 1) Enroll a new fingerprint (with a name attached)
7 2) Verify a fingerprint -> shows whose finger it is
8 3) List everyone enrolled so far
9 4) Exit
10
11 Names are not stored on the sensor itself (the sensor only stores
12 fingerprint templates against numeric IDs), so this script keeps a
13 small local JSON file (fingerprint_names.json) in the same folder,
14 mapping each ID to a name.
15
16 Requirements:
17 pip install pyfingerprint --break-system-packages
18
19 Wiring (R3075 is 3.3V TTL, no level shifter needed):
20 Sensor VCC (red) -> Pi Pin 2 (5V)
21 Sensor GND (black) -> Pi Pin 6 (GND)
22 Sensor RX (white) -> Pi Pin 8 (GPIO14 / TXD)
23 Sensor TX (yellow) -> Pi Pin 10 (GPIO15 / RXD)
24
25 LCD wiring (16x2 with I2C backpack):
26 LCD VCC -> Pi Pin 2 (5V)
27 LCD GND -> Pi Pin 6 (GND)
28 LCD SDA -> Pi Pin 3 (GPIO2 / SDA)
29 LCD SCL -> Pi Pin 5 (GPIO3 / SCL)
30
31 Extra setup for the LCD (in addition to the fingerprint sensor setup):
32 1) sudo raspi-config -> Interface Options -> I2C -> Enable, then reboot
33 2) sudo apt install -y i2c-tools
34 3) i2cdetect -y 1 (note the address shown, usually 0x27 or 0x3F)
35 4) pip install RPLCD smbus2 --break-system-packages
36 5) Set LCD_ADDRESS below to match what i2cdetect showed
37 """
38
39 import sys
40 import time
41 import json
42 import os
43
44 try:
45     from pyfingerprint.pyfingerprint import PyFingerprint
46 except ImportError:
47     print("pyfingerprint is not installed. Run: pip install pyfingerprint --break-system-packages")
48     sys.exit(1)
49
50 # ---- Config ----
51 SERIAL_PORT = '/dev/serial0' # Try '/dev/ttyAMA0' if this doesn't work
52 BAUD_RATE = 57600
53 PASSWORD = 0x00000000 # R3075's actual default password
54
55 NAMES_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'fingerprint_names.json')
56
57 LCD_ADDRESS = 0x27 # change this to match what i2cdetect -y 1 showed (0x27 or 0x3f are common)
58
59 # ----- LCD (16x2, I2C) -----
60
```

```

81
82 lcd = None
83 try:
84     from RPLCD.i2c import CharLCD
85     lcd = CharLCD(i2c_expander='PCF8574', address=LCD_ADDRESS, port=1,
86                   cols=16, rows=2, charmap='A00', auto_linebreaks=True)
87 except Exception as e:
88     print(f"[WARNING] Could not initialize the LCD: {e}")
89     print("The program will still work, just without the display. Check wiring/address/i2cdetect.")
90     lcd = None
91
92
93
94
95 def lcd_write(line1, line2):
96     """Writes two centered lines to the 16x2 LCD. Does nothing if the LCD isn't available."""
97     if lcd is None:
98         return
99     try:
100         lcd.clear()
101         lcd.cursor_pos = (0, 0)
102         lcd.write_string(line1[:16].center(16))
103         lcd.cursor_pos = (1, 0)
104         lcd.write_string(line2[:16].center(16))
105     except Exception as e:
106         print(f"[WARNING] LCD write failed: {e}")
107
108
109 def lcd_show_idle():
110     """The default idle screen: 'Fingerprint' on top, 'Checker' centered below."""
111     lcd_write("Fingerprint", "Checker")
112
113

```

```

114
115 # ----- Name mapping (JSON file) -----
116
117 def load_names():
118     if not os.path.exists(NAMES_FILE):
119         return {}
120     try:
121         with open(NAMES_FILE, 'r') as fh:
122             return json.load(fh)
123     except (json.JSONDecodeError, IOError):
124         return {}
125
126
127 def save_names(names):
128     with open(NAMES_FILE, 'w') as fh:
129         json.dump(names, fh, indent=2)
130
131
132 # ----- Sensor connection -----
133
134 def flush_stale_bytes():
135     """Clears any leftover bytes sitting in the serial buffer."""
136     try:
137         import serial
138         raw = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
139         time.sleep(0.3)
140         raw.reset_input_buffer()
141         raw.reset_output_buffer()
142         raw.close()
143         time.sleep(0.3)
144     except Exception:
145

```

```

122         pass
123
124
125 def connect():
126     """Connects to the sensor and prints basic info."""
127     flush_stale_bytes()
128
129     last_error = None
130     for attempt in range(1, 4):
131         try:
132             f = PyFingerprint(SERIAL_PORT, BAUD_RATE, 0xFFFFFFFF, PASSWORD)
133         except Exception as e:
134             last_error = f"connect error: {e}"
135             time.sleep(0.5)
136             continue
137
138         if f.verifyPassword():
139             print("[OK] Connected to the module successfully!")
140             print(f" Capacity      : {f.getStorageCapacity()} templates")
141             print(f" Stored templates: {f.getTemplateCount()}")
142             return f
143         else:
144             last_error = "password verification returned false"
145             print(f"[RETRY {attempt}/3] Password verification failed, retrying...")
146             time.sleep(0.5)
147
148     print(f"[ERROR] Could not connect after 3 attempts. Last reason: {last_error}")
149     sys.exit(1)
150
151

```

```

52 # ----- Enroll -----
53
54 def enroll(f, names):
55     try:
56         print("\nPlace your finger on the sensor...")
57         while not f.readImage():
58             pass
59         f.convertImage(0x01)
60
61         result = f.searchTemplate()
62         position_number = result[0]
63         if position_number >= 0:
64             existing_name = names.get(str(position_number), "(name not on record)")
65             print(f"[INFO] This finger is already enrolled - {existing_name} (ID #{position_number})")
66             return
67
68         name = input("Enter a name for this finger: ").strip()
69         if not name:
70             print("[ERROR] Name cannot be empty, enrollment cancelled.")
71             return
72
73         print("Remove your finger...")
74         time.sleep(2)
75
76         print("Place the same finger again...")
77         while not f.readImage():
78             pass
79         f.convertImage(0x02)
80
81         if f.compareCharacteristics() == 0:
82             print("[ERROR] The two scans didn't match, try again.")
83

```

```

183         return
184
185     f.createTemplate()
186     position_number = f.storeTemplate()
187
188     names[str(position_number)] = name
189     save_names(names)
190
191     print(f"[OK] Enrolled {name}'s finger! (ID #{position_number})")
192
193 except Exception as e:
194     print(f"[ERROR] Enrollment failed: {e}")
195
196 # ----- Verify -----
197
198 def verify(f, names):
199     try:
200         print("\nPlace your finger on the sensor to verify...")
201         while not f.readImage():
202             pass
203         f.convertImage(0x01)
204
205         result = f.searchTemplate()
206         position_number, accuracy_score = result
207
208         if position_number == -1:
209             print("[NO-MATCH] This finger doesn't match any enrolled ID.")
210             lcd_write("No match", "Try again")
211         else:

```

Shell

Python 3.11.2, 32-bit (root/bin/python3)
 >>>

```

212     name = names.get(str(position_number), "(name not on record for this ID)")
213     print(f"\n>>> Welcome, {name}! <<<")
214     print(f"      (ID #{position_number}, confidence score: {accuracy_score})")
215     lcd_write("Welcome", name)
216
217     time.sleep(3)
218     lcd_show_idle()
219
220 except Exception as e:
221     print(f"[ERROR] Verification failed: {e}")
222     lcd_show_idle()
223
224 # ----- List -----
225
226 def list_enrolled(names):
227     if not names:
228         print("\nNo one is enrolled yet.")
229         return
230     print("\n== Enrolled people ==")
231     for id_str, name in sorted(names.items(), key=lambda x: int(x[0])):
232         print(f"  ID #{id_str}: {name}")
233
234 # ----- Main -----
235
236 def main():
237     print("== R3075 Fingerprint Attendance/ID Tool ==\n")
238     f = connect()

```

```
242 names = load_names()
243 lcd_show_idle()
244
245 while True:
246     print("\nWhat would you like to do?")
247     print(" 1) Enroll a new fingerprint (with a name)")
248     print(" 2) Verify a fingerprint (shows the name)")
249     print(" 3) List all enrolled people")
250     print(" 4) Exit")
251     choice = input("Choice (1-4): ").strip()
252
253     if choice == '1':
254         enroll(f, names)
255     elif choice == '2':
256         verify(f, names)
257     elif choice == '3':
258         list_enrolled(names)
259     elif choice == '4':
260         print("Bye!")
261         break
262     else:
263         print("Invalid choice, try again.")
264
265 if __name__ == '__main__':
266     main()
267
268
Shell
Python 3.3.2, 32-bit (/usr/bin/python)
```

